AUTONOMOUS WORKFLOW RECOVERY

Mend

Project Documentation

A practical guide to the Mend workflow recovery console, its demo features, setup, and path to production.

PROJECT GUIDE · OCTOBER 2026

MONITOR	DIAGNOSE	RECOVER

Autonomous workflow recovery console Project guide · October 2026

1. Overview

Mend is a responsive web application prototype for teams that depend on business automations and software integrations. It presents a single operations console for monitoring workflow health, reviewing failed executions, understanding likely causes, evaluating proposed fixes, and retrying a run after approval.

The concept addresses a common operational problem: a failed API request, changed payload shape, timeout, or rate limit can interrupt a business process and require an engineer to investigate logs manually. Mend's intended recovery lifecycle is:

1. Observe workflow activity and detect a failed or unexpected outcome.
2. Diagnose the likely cause from logs and execution context.
3. Propose a corrective action, such as a payload transformation or retry policy.
4. Validate the fix in isolation, then retry with human approval when appropriate.

Prototype status: The current project is a frontend demo. Its example workflows, incidents, logs, metrics, and diagnosis are seeded UI data. Approval and retry update local demo state; they do not call an AI model or execute real workflows.

2. What the application includes

Overview

The overview is the authenticated home page. It summarizes successful runs, time recovered, auto-heal rate, and the number of incidents awaiting review. It also provides a workflow health list and recent recovery activity.

Workflows

The workflow library lists seeded automations and their source/destination systems, health, success rate, recent run count, and last-run time. Use the search field and status filter to narrow the list. The switch on each row pauses or resumes that workflow in the demo.

Incidents

The recovery center lists detected issues with type, severity, status, and detection time. Search by incident, workflow, or issue type and filter by recovery status. Select an incident to open its details.

An incident detail includes the reported symptoms, an example execution log, the suspected cause and confidence, a suggested transformation, and a timeline. **Approve & retry** simulates validation and recovery: the control briefly enters a validation state and then marks the incident as auto-healed. This is a UI simulation, not a production retry.

Run history

Run history provides a searchable and filterable example execution log. Open a row to inspect a run summary and simplified execution trace. Export is represented as a demo action.

Integrations

The integrations page groups example services by category. Connect/disconnect buttons update local demo state. No provider credentials are requested or stored, and no external service is contacted.

Settings

Settings presents general workspace preferences, recovery behavior, approval policy, notifications, and a security summary. Controls demonstrate the intended experience; configuration is not sent to a backend.

Account access

Use **Create an account** from the sign-in page to open the signup form. Demo sign-in accepts any valid email address and a password of at least six characters. The session flag is stored in this browser's local storage. Select the profile area at the bottom of the sidebar to sign out.

This demo authentication is not an identity or security system. Do not use real credentials.

3. Getting started

Requirements

- Node.js with npm
- A modern browser

Install and run

From the project directory:

```sh npm install npm run dev

Open the local address printed by Vite. If the default port is occupied, Vite may choose another available port; use the address shown in the terminal.

### Build and lint
```

npm run lint npm run build

```
The build runs TypeScript project checks and creates the production frontend bundle in `dist/`. The lint command runs Oxlint.

## 4. Technology and structure

The application uses React and TypeScript, built with Vite. React Router provides client-side routes, Lucide React supplies interface icons, and local storage preserves the demo session and selected workflow, incident, and integration state in the browser.

Main files:

- `src/App.tsx` — routes, page components, seeded data, and demo interactions.
```

- `src/App.css` — responsive Mend interface styles.
- `src/index.css` — global design tokens and base styles.
- `src/main.tsx` — React application entry point.
- `index.html` — document title and metadata.

5. Suggested production architecture

A production implementation would require trusted server-side services around the frontend:

1. **Workflow observer:** accept signed webhooks or execution events; validate source identity and normalize event metadata.
2. **Incident store:** persist runs, logs, payload metadata, approvals, and recovery attempts with access controls and retention policies.
3. **Diagnostic service:** classify errors using deterministic rules first, then optionally use a model with structured output and redacted context.
4. **Patch and policy engine:** generate a constrained proposal, validate it against schemas and allowlists, and enforce approval thresholds. Never execute arbitrary model-generated code directly.
5. **Isolated verifier:** replay a sanitized or representative payload in a test environment and compare output against explicit invariants.
6. **Retry worker:** perform bounded retries with idempotency keys, rate-limit awareness, exponential backoff, and dead-letter handling.
7. **Human approval and audit:** record who approved what, the exact patch version, validation results, and retry outcome.
8. **Provider integrations:** store secrets in a managed secret store, rotate credentials, apply least privilege, and expose revocation controls.

6. Production readiness and safety

Before connecting mission-critical workflows, implement and test:

- Real authentication, authorization, tenant isolation, and secure session management.
- Server-side persistence, encryption, audit logs, backups, and retention controls.
- Verified webhook signatures, request limits, input validation, and secret management.
- Strictly scoped remediation actions, approval policies, dry-run validation, rollback paths, and idempotent retries.
- Monitoring for the recovery service itself, including retry loops, model/provider outages, and alert delivery failures.
- Privacy controls for logs and payloads, including redaction of personal data and credentials before model analysis.
- Tests against malformed payloads, duplicate events, partial failures, rate limits, timeouts, and unsafe proposed changes.

The example success metrics and incident records in this UI are illustrative and should not be interpreted as measured production performance.

7. Current boundaries

The current prototype does not include a backend API, database, actual model provider, webhook receiver, real service integrations, production authentication, durable cross-device storage, or real job execution. The controls demonstrate intended workflows only.

Mend · Project Documentation